

# Informe de análisis de Audio



Presentado por:  
**Silvana Alejandra Gerez**

Profesora:  
**Yanina Escudero**

Instituto:  
**Instituto tecnológico Beltran ISFT 197**

Carrera:  
**Tecnicatura en Ciencia de Datos e  
Inteligencia Artificial**

2° año  
Junio 2026

## **Introduccion**

El presente informe técnico desarrolla el flujo de trabajo estándar para la preparación y análisis de señales de audio digital, aplicado sobre el set de datos *AnalisisTexto16.wav*. En el marco de la asignatura *Técnicas de Procesamiento del Habla para NLP*, el objetivo fundamental es la estandarización y caracterización de señales sonoras como paso crítico previo a su alimentación en modelos de aprendizaje profundo. A través de esta práctica, se exploran los pilares del procesamiento digital de señales (DSP): desde la inspección de metadatos y la normalización de la frecuencia de muestreo hasta la manipulación del rango dinámico mediante recuantización, procesos indispensables para garantizar la integridad y homogeneidad de los datos en tareas de clasificación acústica y reconocimiento de voz.

### **1. Análisis del audio con MediaInfo**

Para conocer los metadatos técnicos del archivo *AnalisisTexto16.wav*, utilizamos la herramienta MediaInfo. Este software permite inspeccionar los encabezados del archivo para determinar su estructura interna sin necesidad de reproducirlo, proporcionando datos críticos sobre la codificación. Identificar el formato, la tasa de bits, la cantidad de canales y la frecuencia de muestreo nativa.

MediaInfo report of "AnálisisTextos.mp3" :

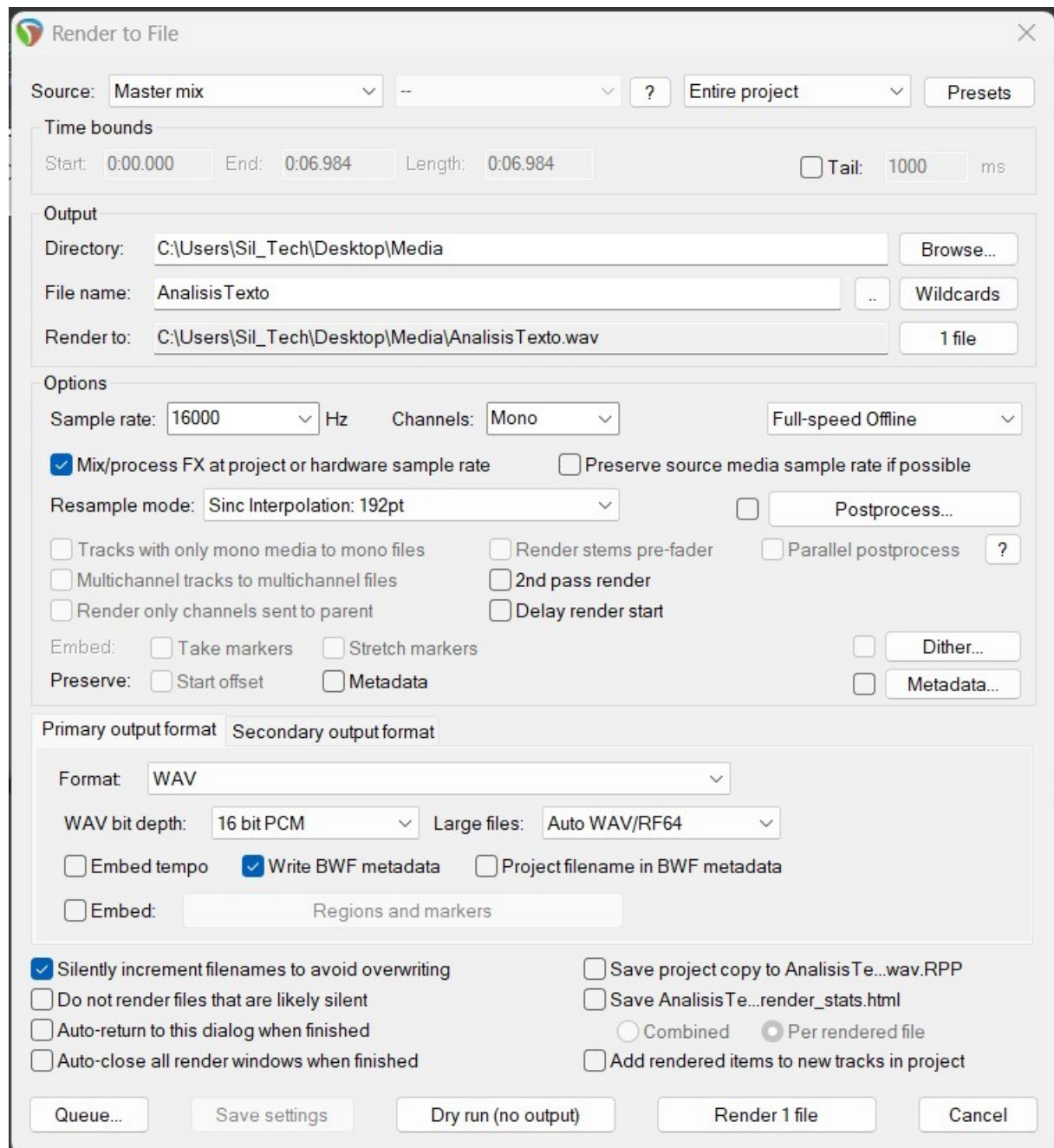
```
General
Complete name      : AnalisisTextos.mp3
Format             : MPEG Audio
File size          : 219 KiB
Duration           : 7 s 12 ms
Overall bit rate mode : Constant
Overall bit rate   : 256 kb/s
Writing library    : LAME?

Audio
Format             : MPEG Audio
Format version     : Version 1
Format profile     : Layer 3
Duration           : 7 s 12 ms
Bit rate mode      : Constant
Bit rate           : 256 kb/s
Channel(s)         : 1 channel
Sampling rate      : 48.0 kHz
Frame rate         : 41.667 FPS (1152 SPF)
Compression mode   : Lossy
Stream size        : 219 KiB (100%) / 219 KiB (100%) / 219 KiB (100%) / 219 KiB (100%)
Writing library    : LAME?
```

## **2. Realización del sampleo (Remuestreo)**

En esta etapa, procedemos a estandarizar la señal para asegurar su compatibilidad con los modelos de aprendizaje profundo (que generalmente operan a 16 kHz). Este proceso de *downsampling* o remuestreo ajusta la cantidad de muestras por segundo del archivo original a los parámetros requeridos por la cátedra. Se realizó mediante el uso de la Herramienta de audio de Reaaper por su renderización <http://www.reaper.fm/download.php>

ingresar desde la barra de tareas del programa a **File** → **Render**



### 3. Análisis del audio post-procesamiento

Tras aplicar el sampleo volvemos a utilizar **MediaInfo** para validar que el archivo resultante cumpla con las especificaciones técnicas exigidas (frecuencia de muestreo ajustada a 16 kHz, formato, etc.).



MediaInfo report of "AnalisisTexto16.wav" :

```
General
Complete name      : AnalisisTexto16.wav
Format             : Wave
Format settings    : PcmWaveformat
File size          : 219 KiB
Duration           : 6 s 985 ms
Overall bit rate mode : Constant
Overall bit rate   : 257 kb/s
Producer          : REAPER
Encoded date       : 2026-06-12 11-52-12

Audio
Format             : PCM
Format settings    : Little / Signed
Codec ID           : 1
Duration           : 6 s 985 ms
Bit rate mode      : Constant
Bit rate           : 256 kb/s
Channel(s)         : 1 channel
Sampling rate      : 16.0 kHz
Bit depth          : 16 bits
Stream size        : 218 KiB (100%) / 218 KiB (100%)
```

**Verificación  
de sampleo**

## 4. Carga de datos y visualización de parámetros

Para iniciar el procesamiento digital, se utilizó la librería **soundfile** de Python, la cual permite la lectura de archivos con extensión **.wav**. La función **sf.read()** extrae dos componentes esenciales: el vector numérico con las muestras de amplitud de la señal y la frecuencia de muestreo de origen ( $F_2$ ).

A continuación, se detalla el fragmento de código implementado para la captura y visualización de las propiedades métricas del archivo **AnalisisTexto16.wav**:

```
# Carga de datos y extracción de variables estructurales
audio, sr = sf.read(ruta_directa)

print("Vector de la señal segmentada:")
print(audio)
print('Largo array:', len(audio))
print('Frecuencia de Muestreo:', sr)
print('Duración:', len(audio) / sr)
```

## Resultados obtenidos:

```
=== DESARROLLO DEL PUNTO 4 =====  
Vector de la señal segmentada:  
[ 0.      0.      0.      ...  0.00646973 -0.0062561  
 -0.01470947]  
Largo array: 111760  
Frecuencia de Muestreo: 16000  
Duración: 6.985  
=====
```

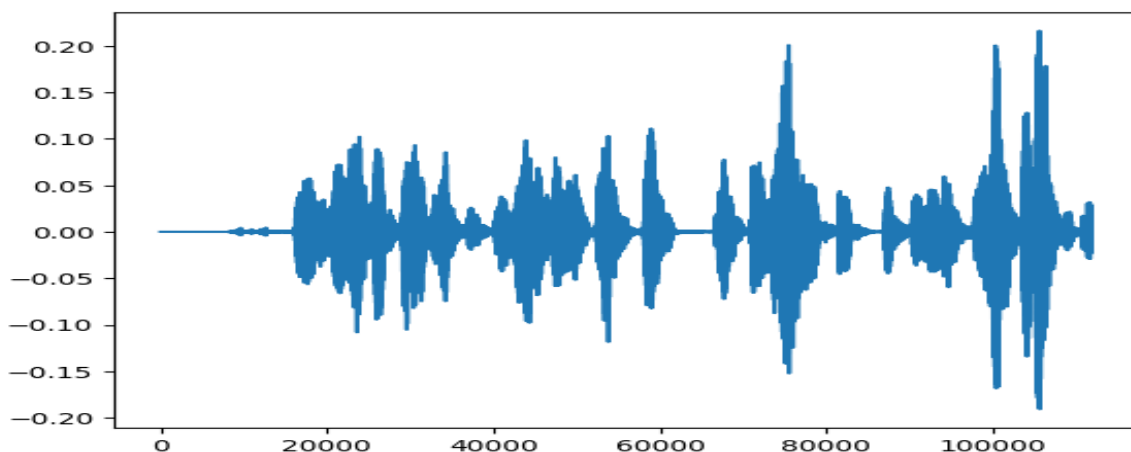
## 5. Impresión de la señal sonora (Gráfico de forma de onda)

La representación visual de la señal en el dominio del tiempo (forma de onda) permite analizar la evolución de la amplitud a lo largo de las muestras. Para ello, se empleó la librería matemática `matplotlib.pyplot`. El método `plt.plot(audio)` toma los valores del vector numérico y los proyecta de forma continua en un eje bidimensional.

### # Código para la generación del gráfico de forma de onda

```
# Código para la generación del gráfico de forma de onda  
plt.plot(audio)  
plt.show()
```

## Visualización de la señal continua:

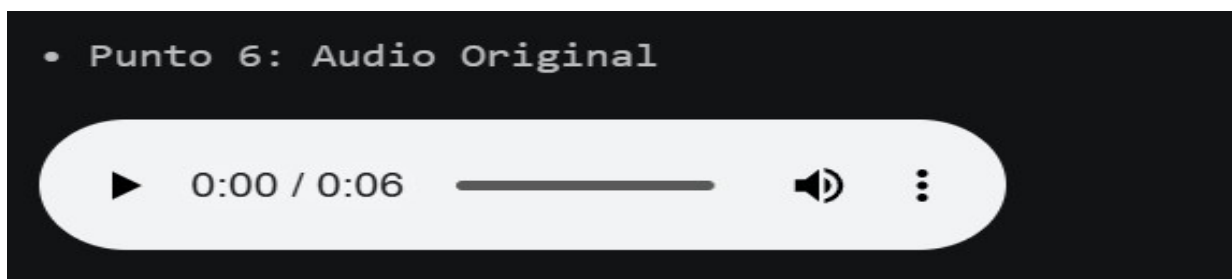


## 6. Reproducción de la señal original

Para la reproducción de la señal se utiliza el módulo `IPython.display.Audio`. Este componente genera una interfaz interactiva dentro del entorno de desarrollo (una barra con controles de reproducción) que permite enviar el vector original a la placa de sonido del sistema a la velocidad nativa especificada por la variable `sr` (en 16 bits de resolución).

```
# Interfaz de reproducción nativa sin alteraciones
display(Audio(audio, rate=sr))
```

*vista de la interfaz interactiva:*



## 7. Modificación de la frecuencia de muestreo (Duración y tono)

En este apartado se alteró intencionalmente el parámetro de velocidad de reproducción (`rate`) dentro de la función de audio, modificando la cantidad de muestras que el sistema procesa por segundo. Las variaciones de tiempo y tono se rigen por la física de las señales digitales:

[# Alteración de la velocidad y duración por software](#)

```
# Alteración de la velocidad y duración por software
# Caso 7a: Incremento de duración (0.3 veces la tasa original)
display(Audio(audio, rate=sr * 0.3))

# Caso 7b: Reducción de duración (Al doble de la tasa original)
display(Audio(audio, rate=sr * 2))
```

## Explicación del fenómeno acústico:

- **Punto 7a (Mayor duración):** Al multiplicar la frecuencia por un factor menor a la unidad ( $sr * 0.3$ ), se obliga al sistema a reproducir menos muestras por segundo. Como consecuencia, el audio se distribuye en una ventana de tiempo considerablemente más larga (**dura más tiempo**) y las frecuencias se desplazan hacia el extremo inferior del espectro, provocando un **tono marcadamente más grave**.
- **Punto 7b (Menor duración):** Al duplicar la tasa de muestreo ( $sr * 2$ ), el vector de datos se procesa a doble velocidad. Esto provoca que el archivo final **dure menos tiempo** y que las frecuencias se compriman en el tiempo, elevando el tono de la voz hacia un **perfil mucho más agudo**.

## Vista interactiva de los audios modificados

- Punto 7a: Mayor duración (Menor velocidad / Tono grave)

▶

0:00 / 0:23

🔊

⋮

- Punto 7b: Menor duración (Mayor velocidad / Tono agudo)

▶

0:00 / 0:03

🔊

⋮



## 8. Baja calidad del audio y reproducción de la señal

Para simular una pérdida severa de calidad sin alterar la frecuencia de muestreo temporal, se procedió a realizar un proceso de **recuantización (reducción de la profundidad de bits)**.

El código provisto por la cátedra aplica una transformación lineal sobre la amplitud y fuerza un casteo de tipo estructural hacia enteros de 8 bits:

### # Proceso matemático de recuantización a 8 bits

```
# Proceso matemático de recuantización a 8 bits
y = (audio * 2**3).astype(np.int8)
display(Audio(y, rate=sr))
```

### Explicación técnica del proceso:

El archivo original posee una resolución estándar de 16 bits, lo que provee un rango dinámico de  $2^{16}$  (65.536) niveles posibles para dibujar la onda de forma precisa.

Al multiplicar el vector por  $2^3$  (reducir su escala efectiva) y aplicarle la instrucción `.astype(np.int8)`, se trunca la precisión decimal y se obliga a la señal a encasillarse en un contenedor de apenas 8 bits (256 niveles lineales discretos).

Al despojar a la onda de su precisión matemática original, el error de redondeo se transforma en un **ruido de cuantización** severo. Acústicamente, este proceso degrada la fidelidad de la señal original, traduciéndose en una pérdida notoria de claridad y en la aparición de un siseo áspero, distorsionado y que llega a enmascarar la voz.

Vista interactiva del reproductor degradado a 8 bits:

- Punto 8: Baja calidad de audio (Conversión a 8 bits)

▶ 0:06 / 0:06

🔊

⋮